

Q1 Student Management System [BL3 – Apply]

10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
START
CLASS student
  PRIVATE
    name
    rollno
    marks
  PUBLIC
    METHOD acceptdetails()
      INPUT name
      INPUT rollno
      INPUT marks
    END METHOD
    METHOD calculateaverage()
      .
    METHOD displayresult()
      average->calculateaverage()
      IF average>=50 THEN
        PRINT "pass"
      ELSE
        PRINT "fail"
      END IF
    END METHOD
END CLASS
MAIN
  create object s of student
  CALL s.acceptdetails()
  average->s.calculateaverage()
  PRINT "Average marks=" average
  CALL s.displayresult()
END
STOP
```

Q2 Library Book Management [BL3 – Apply]

10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
start
class book
  private
    title
    author
    isbn
    isavailable
  public
    method setdetails(t,a,i)
      title<- t
      author<-a
      isbn<-i
      isavailable<-true
    end method
    method issuebook()
      if isavailable=true then
        isavailable<=false
        print "book issued successfully"
      else
        print "book is not available"
      endif
    end method
    method returnbook()
      isavailable<-true
      print "book returned successfully"
    end method
    method gettitle()
      return title
    end method
    method getisbn()
      return isbn
    end method
    method getavailability()
      return isavailable
    end method
    method displaydetails()
```

```

        print "title",gettitle()
        print "author",getauthor()
        print "isbn",getisbn()
        print "availbale",getavailability()
    end method
end class
main
    create book b1
    create book b2
    create book b3
    call b1.setdetails("java","james","101")
    call b2.setdetails("python","guido","102")
    call b3.setdetails("C++","bjarne","103")
    call b1.issuebook()
    call b2.issuebook()
    call b1.returnbook()
    call b1.displaydetails()

    call b2.displaydetails()
    call b3.displaydetails()
end
stop

```

Q3 Employee Salary Calculation [BL3 – Apply]

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empid, name, basicPay, designation). Implement methods to: •

Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```

start
class employee
    private
        empid
        name
        basicpay
        designation
    public
        method acceptdetails()
            input empid

```

```

        input name
        input designation
        input basicpay
    end method
    method calculategs()
        hra<-basicpay*0.20
        da<-basicpay*0.10
        grosssalary<-basicpay+hra+da
        return grosssalary
    end method

    method calculatens()
        grosssalary<-calculategs()
        pf<-basicpay*0.12
        tax<-grosssalary*0.05
        netsalary<-grosssalary-pf-tax
        return netsalary
    end method

    method display()
        print "pay slip"
        print "employee id",empid
        print "name",name
        print "designation",designation
        print "basic pay",basicpay
        print "gross salary",calculategs()
        print "gross salary",calculategs()
        print "net salary",calculatens()
    end class

main
    create employee e
    call e.acceptdetails()
    call e.display()
end
stop

```

Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]

10 mark

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```

start
class shape
    method calculatearea()
        print "area calculation"
    end method
end class

class circle extends shape
    declare radius
    method calculatearea()
        input radius
        area<-3.14*radius*radius
        print "area of circle:",area
    end method

```

```
end class
```

```
class rectangle extends shape
  declare length
  declare breadth
  method calculatearea()
    input length
    input breadth
    area<=length*breadth
    print "area of rectangle:", area
  end method
end class
```

```
class triangle extends shape
  declare base
  declare height
  method calculatearea()
    input base
    input height
    area<=(base*height)/2
    print "area of triangle", area
  end method
end class
```

```
main
```

```
  create shape s
```

```
main
```

```
  create shape s
  s<-new circle
  call s.calculatearea()
  s<-new rectangle
  call s.calculatearea()
  s<-new triangle
  call s.calculatearea()
```

```
end
```

```
stop
```

Q5 5. Vehicle Registration System [BL3 – Apply]

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
start
class vehicle
  regno
  owner
  fueltype
  method setdetails(r,o,f)
    regno<-r
    owner<-o
    fueltype<-f
  end method
  method displaydetails()
    print "registration no", regno
    print "owner", owner
    print "fuel type",fuel type
  end method
end class

class twowheeler extends vehicle
  enginecapacity
  method setenginecap(ec)
    enginecapacity<-ec
  end method
  method displaydettails()
    print"two wheeler"
    print "registration no", regno
    print 'owner', owner
    print "fuel type", fueltype
    print "capacity", enginecapacity
  end method
end class

class fourwheeler extends vehicle
  seatingcap
  acstatus
  method setfourwheelerdetails(sc,ac)

    seatingcap<-sc
    acstatus <-ac
  end method
  method displaydetails()
    print "four wheeler"
    print "reaistration no". reano
```

```

        print "owner", owner
        print "fuel type", fueltype
        print "seating capacity", seatingcap
        print "ac status", acstatus
    end method
end class

main
    create twowheeler t
    call t.setdetails("TN08AV7377", "rahul", "petrol")
    call t.setenginecap(150)
    create fourwheeler f
    call f.setdetails("TN093fg6468", "meera", "diesel")
    call f.fourwheelerdetails(5, "yes")
    call t.displaydetails()
    call f.displaydetails()
end
stop

```

Q6 6. Bank Account Hierarchy [BL4 – Analyze]

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```

start
class bankaccount
    accno
    balance
    holder
    method setdetails(a,b,h)
        accno<-a
        balance<-b
        holder<-h
    end method

```

```
end class
class savingsaccount extends bankaccount
    interestrate
    method calculateinterest()
        interest<=-balance*interestrate/100
        balance<=-balance+interest
    end method
    method withdraw(amount)
        if amount<=balance then
            balance<=-balance-amount
            print "withdraw successful"
        else
            print "insufficient"
        endif
    end method
end class
class currentaccount extends bankaccount
    overdraftlimit
    method withdraw(amount)
        if amount<=balance+overdraftlimit then
            balance<=-balance-amount
            print "withdrawal successful"
        else
            print "overdraft limit exceeded"
        endif
    end method
end class
main
    create savingsaccount s
    call s.setdetails(101,1000,"rahul")
    s.interestrate<-5
    call s.calculateinterest()

    call s.withdraw(3000)
    create currentaccount c
    call c.setdetails(102,5000,"meera")
    c.overdraftlimit<-2000
    call c.withdraw(6000)
end
stop
```

Q7 7. Hospital Patient Record System [BL4 – Analyze]

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
start
class patient
  private
    diagnosis
    prescription
  public
    patientid
    patientname
    method setdetails(id,name,d,p)
      patientid <- id
      patientname <- name
      diagnosis <- d
      prescription <- p
    end method
    method generatebill()
      print "generate bill"
    end method
end class
class inpatient extends patient
  wardnumber
  admissiondate
  dailycharges
  numberofdays
  method generatebill()
    bill <- dailycharges*numberofdays
    print "inpatient bill", bill
  end method
end class

class outpatient extends patient
  appointmentdate
  consultationfee
  method generatebill()
    bill <- consultationfee
    print "outpatient bill", bill
  end method
end class
```

```

main
  create inpatient ip
  call ip.setdetail(101,"rahul","fever","medicine")
  ip.wardnumber<-12
  ip.admissiondate<-"10-07-2026"
  ip.dailycharges<-2000
  ip.numberofdays<-5
  call ip.generatebill()
  create outpatient op
  call op.setdetails(102,"meera","cold","tablet")
  op.appointmentdate<-"15-07-2026"
  op.consultationfee<-500
  call op.generatebill()
end
stop

```

Q8 8. E-Commerce Product Catalog [BL4 – Analyze]

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```

start
class product
  productid
  productname
  price
  method setdetails(id,name,p)
    productid<-id
    productname<-name
    price<-p
  end method
  method applydiscount()

```

```

        print "discount applied"
    end method
end class

class electronics extends product
    warranty
    brand
    voltage
    method applydiscount()
        discountprice<-price-(price*10/100)
        print "electronics discount:", discountprice
    end method
end class

class clothing extends product
    size
    fabric
    gender
    method applydiscount()
        discountprice<-price-(price*20/100)
        print "clothing discount price", discountprice
    end method
end class

class grocery extends product
    expirydate
    weight
    method applydiscount()
        discountprice<-price-(price*5/100)
        print "grocery discount price", discountprice
    end method
end class

main
    create product p
    p<- new electronics
    call p.setdetails(101,"laptop",50000)
    call p.applydiscount()

    p<- new clothing
    call p.setdetails(102,"shirt",2000)
    call p.applydiscount()

    p<-new grocery
    call p.setdetails(103,"rice",800)
    call p.applydiscount()
end
stop

```

Q9 9. University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```
start
class staff
  private
    staffid
  protected
    name'
    basicsalary
  method setdetails(id,n,salary)
    staffid<-id
    staffid<-id
    name<-n
    basicsalary<-salary
  end method
end class

class teachingstaff extends staff
  subject
  method calculateallowance()
    allowance<-basicsalary*0.20
    print "teaching staff allowance", allowance
  end method
end class

class professor extends teachingstaff
  researchallowance
  method calculateallowance()
    aloowance<-(basicsalay*0.20)+researchallowance
    print "professor allowance", allowance
  end method
end class

class lecturer extends teachigstaff
  seminarallowance
  method calculateallowance()
    allowance<- (basicsalay*0.15)+seminarallowance
    print "lecturer allowance:", allowance
```

```

    end method
end class
class nonteachingstaff extends staff
    department
    method calculateallowance()
        allowance<-basicsalary*0.10
        print" non teaching staff allowance", allowance
    end method
end class

main

    create professor
    call p.setdetails(101,"rahul",66000)
    p.researchallowance<-5000
    call p.calculateallowance()
    create lecturer l
    call l.setdetails(102,"meera",40000)
    l.seminarallowance<-2000
    call l.calculateallowance()

    create nonteachigstaff n
    call n.setdetails(103,"amit",35000)
    call n.calculateallowance()

end
stop

```

Q10 10. Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```

start
class smartdevice
    devicename
    method setdevice(name)
        device<-name
    end method
    method executecommand(cmd)
        print "executing comand"
    end method
end class

```

```

class smartlight extends smartdevice
  method executecommand(cmd)
    if cmd="on" then
      print "light is on"
    else if cmd="off" then
      print "light turned off"
    else if cmd="status" then
      print "light status displayed"
    end if
  end method
end class

class smartthermostat extends smartdevice()
  method executecommand(cmd)
    if cmd="on" then
      print "thermostat turned on"
    else if cmd="off" then
      print "thermostat turned off"
    else if cmd='status' then
      print "temperature displayed"
    endif
  end method
end class

class smartlock extends smartdevice
  private
    pin
  method setpin(p)
    pin<-p
  end method
  method executecommand(cmd)
    if cmd="on" then
      print "door locked"
    else if cmd="off" then
      print "door unlocked"
    else if cmd="status" then
      print "lock status displayed"
    end if
  end method
end class

main
  create smartdevice d
  d<-new smartlight
  call d.executecommand("on")
  d<-new smartthermostat
  call d.executecommand("status")
  d<-new smartlock
  call d.executecommand("off")
end
stop

```